```
package main

import (
 "os"

 "eqrcp/cmd"
)

func main() {
 if err := cmd.Execute(); err != nil {
  os.Exit(1)
 }
}
package application

type Flags struct {
 Quiet             bool
 KeepAlive         bool
 ListAllInterfaces bool
 Port              int
 Path              string
 Interface         string
 Bind              string
 FQDN              string
 Zip               bool
 Config            string
 Browser           bool
 Secure            bool
 TlsCert           string
 TlsKey            string
 Output            string
 Reversed          bool
}

type App struct {
 Flags Flags
 Name  string
}

func New() App {
 return App{
  Name:  "eqrcp",
  Flags: Flags{},
 }
}
package cmd

import (
 "eqrcp/application"
 "github.com/spf13/cobra"
```

```
)

var app application.App

func init() {
 app = application.New()
 rootCmd.AddCommand(sendCmd)
 rootCmd.AddCommand(receiveCmd)
 rootCmd.AddCommand(chatCmd)
 rootCmd.AddCommand(configCmd)
 rootCmd.AddCommand(versionCmd)
 rootCmd.AddCommand(completionCmd)
 rootCmd.AddCommand(desktopCmd)
 desktopCmd.AddCommand(desktopShareCmd)
 desktopCmd.AddCommand(desktopReceiveCmd)
 desktopCmd.AddCommand(desktopChatCmd)
 desktopCmd.AddCommand(desktopInstallCmd)
 desktopCmd.AddCommand(desktopStatusCmd)
 desktopCmd.AddCommand(desktopStartupEnableCmd)
 desktopCmd.AddCommand(desktopStartupDisableCmd)
 desktopCmd.AddCommand(desktopStartupStatusCmd)
 desktopCmd.AddCommand(desktopUninstallCmd)
 desktopCmd.AddCommand(desktopAgentCmd)
 desktopCmd.AddCommand(desktopAgentStartCmd)
 desktopCmd.AddCommand(desktopAgentRestartHelperCmd)
 desktopCmd.AddCommand(desktopAgentHistoryClearCmd)
 desktopCmd.AddCommand(desktopAgentStopCmd)
 desktopCmd.AddCommand(desktopAgentStopCurrentCmd)
 desktopCmd.AddCommand(desktopAgentStatusCmd)
 desktopCmd.AddCommand(desktopAgentOpenCmd)
 desktopCmd.AddCommand(desktopAgentOpenCurrentCmd)
 configCmd.AddCommand(migrateCmd)
 desktopAgentCmd.Flags().BoolP("background", "B", false, "start the desktop agent in the backgrou
 desktopAgentStartCmd.Flags().BoolP("background", "B", false, "start the desktop agent in the bac
 // Global command flags
 rootCmd.PersistentFlags().BoolVarP(&app.Flags.Quiet, "quiet", "q", false, "only print errors")
 rootCmd.PersistentFlags().BoolVarP(&app.Flags.KeepAlive, "keep-alive", "k", false, "keep server
 rootCmd.PersistentFlags().BoolVarP(&app.Flags.ListAllInterfaces, "list-all-interfaces", "l", fal
```

```
 // Receive command flags
 receiveCmd.PersistentFlags().StringVarP(&app.Flags.Output, "output", "o", "", "output directory
}

// The root command (‘eqrcp‘) is like a shortcut of the ‘send‘ command
var rootCmd = &cobra.Command{
 Use:           "eqrcp",
 Args:          cobra.MinimumNArgs(1),
 RunE:          sendCmdFunc,
 SilenceErrors: true,
 SilenceUsage:  true,
}

// Execute the root command
func Execute() error {
 if err := rootCmd.Execute(); err != nil {
  rootCmd.PrintErrf("Error: %v\nRun ‘eqrcp help‘ for help.\n", err)
  return err
 }
 return nil
}
package cmd

import (
 "fmt"

 "eqrcp/body"
 "eqrcp/config"
 "eqrcp/logger"
 "eqrcp/qr"
 "github.com/eiannone/keyboard"

 "eqrcp/server"
 "github.com/spf13/cobra"
)

func sendCmdFunc(command *cobra.Command, args []string) error {
 log := logger.New(app.Flags.Quiet)
 body, err := body.FromArgs(args, app.Flags.Zip)
 if err != nil {
  return err
 }
 cfg, err := config.New(app)
 if err != nil {
  return err
 }
 srv, err := server.New(&cfg)
 if err != nil {
  return err
 }
```

```
 // Sets the body
 srv.Send(body)
 log.Print(‘Scan the following URL with a QR reader to start the file transfer, press CTRL+C or "q"
 log.Print(srv.SendURL)
 if err := qr.RenderString(srv.SendURL, cfg.Reversed); err != nil {
  return err
 }
 if app.Flags.Browser {
  if err := srv.DisplayQR(srv.SendURL); err != nil {
   return err
  }
 }
 if err := keyboard.Open(); err == nil {
  defer func() {
   keyboard.Close()
  }()
  go func() {
   for {
    char, key, _ := keyboard.GetKey()
    if string(char) == "q" || key == keyboard.KeyCtrlC {
     srv.Shutdown()
    }
   }
  }()
 } else {
  log.Print(fmt.Sprintf("Warning: keyboard not detected: %v", err))
 }
 if err := srv.Wait(); err != nil {
  return err
 }
 return nil
}

var sendCmd = &cobra.Command{
 Use:     "send",
 Short:   "Send a file(s) or directories from this host",
 Long:    "Send a file(s) or directories from this host",
 Aliases: []string{"s"},
 Example: ‘# Send /path/file.gif. Webserver listens on a random port
eqrcp send /path/file.gif
# Shorter version:
eqrcp /path/file.gif
# Zip file1.gif and file2.gif, then send the zip package
eqrcp /path/file1.gif /path/file2.gif
# Zip the content of directory, then send the zip package
eqrcp /path/directory
# Send file.gif by creating a webserver on port 8080
eqrcp --port 8080 /path/file.gif
‘,
 Args: cobra.MinimumNArgs(1),
```

```
 RunE: sendCmdFunc,
}
package cmd

import (
 "fmt"

 "eqrcp/config"
 "eqrcp/logger"
 "eqrcp/qr"
 "eqrcp/server"
 "github.com/eiannone/keyboard"
 "github.com/spf13/cobra"
)

func receiveCmdFunc(command *cobra.Command, args []string) error {
 log := logger.New(app.Flags.Quiet)
 // Load configuration
 cfg, err := config.New(app)
 if err != nil {
  return err
 }
 // Create the server
 srv, err := server.New(&cfg)
 if err != nil {
  return err
 }
 // Sets the output directory
 if err := srv.ReceiveTo(cfg.Output); err != nil {
  return err
 }
 // Prints the URL to scan to screen
 log.Print(‘Scan the following URL with a QR reader to start the file transfer, press CTRL+C or "q"
 log.Print(srv.ReceiveURL)
 // Renders the QR
 if err := qr.RenderString(srv.ReceiveURL, cfg.Reversed); err != nil {
  return err
 }
 if app.Flags.Browser {
  if err := srv.DisplayQR(srv.ReceiveURL); err != nil {
   return err
  }
 }
 if err := keyboard.Open(); err == nil {
  defer func() {
   keyboard.Close()
  }()
  go func() {
   for {
    char, key, _ := keyboard.GetKey()
```

```
    if string(char) == "q" || key == keyboard.KeyCtrlC {
     srv.Shutdown()
    }
   }
  }()
 } else {
  log.Print(fmt.Sprintf("Warning: keyboard not detected: %v", err))
 }
 if err := srv.Wait(); err != nil {
  return err
 }
 return nil
}

var receiveCmd = &cobra.Command{
 Use:     "receive",
 Aliases: []string{"r"},
 Short:   "Receive one or more files",
 Long:    "Receive one or more files. The destination directory can be set with the config wizard, o
 Example: ‘# Receive files in the current directory
eqrcp receive
# Receive files in a specific directory
eqrcp receive --output /tmp
‘,
 RunE: receiveCmdFunc,
}
package cmd

import (
 "fmt"

 "eqrcp/config"
 "eqrcp/logger"
 "eqrcp/qr"
 "eqrcp/server"
 "github.com/eiannone/keyboard"
 "github.com/spf13/cobra"
)

func chatCmdFunc(command *cobra.Command, args []string) error {
 log := logger.New(app.Flags.Quiet)
 cfg, err := config.New(app)
 if err != nil {
  return err
 }
 srv, err := server.New(&cfg)
 if err != nil {
  return err
 }
 log.Print(‘Scan the following URL with a QR reader to join the chat session, press CTRL+C or "q" t
```

```
 log.Print(srv.ChatJoinURL())
 if err := qr.RenderString(srv.ChatJoinURL(), cfg.Reversed); err != nil {
  return err
 }
 if app.Flags.Browser {
  if err := srv.DisplayChat(); err != nil {
   return err
  }
 } else {
  if err := srv.Chat(); err != nil {
   return err
  }
 }
 if err := keyboard.Open(); err == nil {
  defer func() {
   keyboard.Close()
  }()
  go func() {
   for {
    char, key, _ := keyboard.GetKey()
    if string(char) == "q" || key == keyboard.KeyCtrlC {
     srv.Shutdown()
    }
   }
  }()
 } else {
  log.Print(fmt.Sprintf("Warning: keyboard not detected: %v", err))
 }
 return srv.Wait()
}

var chatCmd = &cobra.Command{
 Use:     "chat",
 Short:   "Start a browser chat session with another device",
 Long:    "Start a browser chat session where this host and a scanned mobile device can exchange tex
 Aliases: []string{"c"},
 Example: '# Start a chat session and print a QR code
eqrcp chat
# Start a chat session and open the desktop browser interface
eqrcp chat --browser',
 Args: cobra.NoArgs,
 RunE: chatCmdFunc,
}
package cmd

import (
 "fmt"

 "eqrcp/config"
 "github.com/spf13/cobra"
```

```
)

func configCmdFunc(command *cobra.Command, args []string) error {
 return config.Wizard(app)
}

var configCmd = &cobra.Command{
 Use:     "config",
 Short:   "Configure eqrcp",
 Long:    "Run an interactive configuration wizard for eqrcp. With this command you can configure wh
 Aliases: []string{"c", "cfg"},
 RunE:    configCmdFunc,
}

var migrateCmd = &cobra.Command{
 Use:   "migrate",
 Short: "Migrate the legacy configuration file",
 Long:  "Migrate the legacy JSON configuration file to the new YAML format",
 Run: func(cmd *cobra.Command, args []string) {
  ok, err := config.Migrate(app)
  if err != nil {
   fmt.Println("error while migrating the legacy JSON configuration file:", err)
  }
  if ok {
   fmt.Println("Legacy JSON configuration file has been successfully deleted")
  }
 },
}
package cmd

import (
 "fmt"

 "eqrcp/version"
 "github.com/spf13/cobra"
)

var versionCmd = &cobra.Command{
 Use:   "version",
 Short: "Print version number and build information.",
 Run: func(c *cobra.Command, args []string) {
  fmt.Println(version.String())
 },
}
package cmd

import (
 "os"

 "github.com/spf13/cobra"
```

```
)

// completionCmd represents the completion command
var completionCmd = &cobra.Command{
 Use:   "completion [bash|zsh|fish|powershell]",
 Short: "Generate completion script",
 Long: ‘To load completions:

Bash:

$ source <(eqrcp completion bash)

# To load completions for each session, execute once:
Linux:
  $ eqrcp completion bash > /etc/bash_completion.d/eqrcp
MacOS:
  $ eqrcp completion bash > /usr/local/etc/bash_completion.d/eqrcp

Zsh:

# If shell completion is not already enabled in your environment you will need
# to enable it.  You can execute the following once:

$ echo "autoload -U compinit; compinit" >> ~/.zshrc

# To load completions for each session, execute once:
$ eqrcp completion zsh > "${fpath[1]}/_eqrcp"

# You will need to start a new shell for this setup to take effect.

Fish:

$ eqrcp completion fish | source

# To load completions for each session, execute once:
$ eqrcp completion fish > ~/.config/fish/completions/eqrcp.fish
‘,
 DisableFlagsInUseLine: true,
 ValidArgs:             []string{"bash", "zsh", "fish", "powershell"},
 Args:                  cobra.MatchAll(cobra.ExactArgs(1), cobra.OnlyValidArgs),
 RunE: func(cmd *cobra.Command, args []string) error {
  switch args[0] {
  case "bash":
   if err := cmd.Root().GenBashCompletion(os.Stdout); err != nil {
    return err
   }
  case "zsh":
   if err := cmd.Root().GenZshCompletion(os.Stdout); err != nil {
    return err
   }
```

```
  case "fish":
   if err := cmd.Root().GenFishCompletion(os.Stdout, true); err != nil {
    return err
   }
  case "powershell":
   if err := cmd.Root().GenPowerShellCompletion(os.Stdout); err != nil {
    return err
   }
  }
  return nil
 },
}
package cmd

import (
 "fmt"

 "eqrcp/application"
 "eqrcp/config"
 "eqrcp/version"
 "github.com/spf13/cobra"
)

var desktopCmd = &cobra.Command{
 Use:   "desktop",
 Short: "Desktop integration helpers",
 Long:  "Desktop integration helpers for file manager context menus and other non-terminal launchers
}

var desktopShareCmd = &cobra.Command{
 Use:   "share <file-or-directory> [file-or-directory...]",
 Short: "Share selected files or directories from a desktop launcher",
 Long:  "Share selected files or directories from a desktop launcher. This command opens the QR code
 Example: ‘# Share one file from a desktop launcher
eqrcp desktop share /path/file.txt
# Share multiple selected paths
eqrcp desktop share /path/file.txt /path/photo.jpg
# Share a directory
eqrcp desktop share /path/directory‘,
 Args: cobra.MinimumNArgs(1),
 RunE: func(command *cobra.Command, args []string) error {
  app.Flags.Browser = desktopBrowserPreference(app.Flags, true)
  return sendCmdFunc(command, args)
 },
}

var desktopReceiveCmd = &cobra.Command{
 Use:   "receive [directory]",
 Short: "Receive files into a directory from a desktop launcher",
 Long:  "Receive files into a directory from a desktop launcher. This command opens the QR code in a
```

```
 Example: ‘# Receive files into a selected directory
eqrcp desktop receive /path/directory
# Receive files into the configured output directory or current directory
eqrcp desktop receive‘,
 Args: cobra.RangeArgs(0, 1),
 RunE: func(command *cobra.Command, args []string) error {
  app.Flags.Browser = desktopBrowserPreference(app.Flags, true)
  if output, ok := desktopReceiveOutput(args); ok {
   app.Flags.Output = output
  } else if app.Flags.Output == "" {
   app.Flags.Output = desktopOutputPreference(app.Flags)
  }
  return receiveCmdFunc(command, args)
 },
}

var desktopChatCmd = &cobra.Command{
 Use:   "chat",
 Short: "Start a chat session from a desktop launcher",
 Long:  "Start a browser-based chat session from a desktop launcher. This command opens the chat int
 Example: ‘# Start a chat session
eqrcp desktop chat‘,
 Args: cobra.NoArgs,
 RunE: func(command *cobra.Command, args []string) error {
  app.Flags.Browser = desktopBrowserPreference(app.Flags, true)
  return chatCmdFunc(command, args)
 },
}

func desktopReceiveOutput(args []string) (string, bool) {
 if len(args) == 0 {
  return "", false
 }
 return args[0], true
}

func desktopBrowserPreference(flags application.Flags, fallback bool) bool {
 settingsApp := application.New()
 settingsApp.Flags = flags
 settings, err := config.ReadDesktopSettings(settingsApp)
 if err != nil {
  return fallback
 }
 return settings.Browser
}

func desktopOutputPreference(flags application.Flags) string {
 settingsApp := application.New()
 settingsApp.Flags = flags
 settings, err := config.ReadDesktopSettings(settingsApp)
```

```
 if err != nil || settings.Output == "" {
  return config.DefaultDesktopOutputDirectory()
 }
 return settings.Output
}

var desktopInstallCmd = &cobra.Command{
 Use:   "install",
 Short: "Install desktop context menu entries",
 Long:  "Install desktop context menu entries for the current user.",
 RunE: func(command *cobra.Command, args []string) error {
  if err := installDesktopIntegration(); err != nil {
   return err
  }
  fmt.Fprintln(command.OutOrStdout(), "Desktop context menu entries installed.")
  return nil
 },
}

var desktopStatusCmd = &cobra.Command{
 Use:   "status",
 Short: "Show desktop context menu integration status",
 Long:  "Show desktop context menu integration status for the current user.",
 RunE: func(command *cobra.Command, args []string) error {
  status, err := desktopIntegrationStatus()
  if err != nil {
   return err
  }
  fmt.Fprintf(command.OutOrStdout(), "%s\n%s", version.String(), status)
  return nil
 },
}

var desktopStartupEnableCmd = &cobra.Command{
 Use:   "startup-enable",
 Short: "Start the desktop agent at login",
 Long:  "Register the desktop integration agent to start when the current user logs in.",
 RunE: func(command *cobra.Command, args []string) error {
  if err := installDesktopStartup(); err != nil {
   return err
  }
  fmt.Fprintln(command.OutOrStdout(), "Desktop agent startup enabled.")
  return nil
 },
}

var desktopStartupDisableCmd = &cobra.Command{
 Use:   "startup-disable",
 Short: "Stop starting the desktop agent at login",
 Long:  "Remove the current-user login startup registration for the desktop integration agent.",
```

```
 RunE: func(command *cobra.Command, args []string) error {
  if err := uninstallDesktopStartup(); err != nil {
   return err
  }
  fmt.Fprintln(command.OutOrStdout(), "Desktop agent startup disabled.")
  return nil
 },
}

var desktopStartupStatusCmd = &cobra.Command{
 Use:   "startup-status",
 Short: "Show desktop agent startup status",
 Long:  "Show whether the desktop integration agent is registered to start when the current user log
 RunE: func(command *cobra.Command, args []string) error {
  status, err := desktopStartupStatus()
  if err != nil {
   return err
  }
  fmt.Fprintf(command.OutOrStdout(), "%s\n%s", version.String(), status)
  return nil
 },
}

var desktopUninstallCmd = &cobra.Command{
 Use:   "uninstall",
 Short: "Uninstall desktop context menu entries",
 Long:  "Uninstall desktop context menu entries created by eqrcp.",
 RunE: func(command *cobra.Command, args []string) error {
  if err := uninstallDesktopIntegration(); err != nil {
   return err
  }
  fmt.Fprintln(command.OutOrStdout(), "Desktop context menu entries uninstalled.")
  return nil
 },
}
package cmd

import (
 "context"
 "encoding/json"
 "fmt"
 "html/template"
 "io"
 "net"
 "net/http"
 "net/url"
 "os"
 "os/exec"
 "path/filepath"
 "runtime"
```

```
 "strconv"
 "strings"
 "sync"
 "time"

 "eqrcp/application"
 "eqrcp/body"
 "eqrcp/config"
 "eqrcp/server"
 "eqrcp/version"
 "github.com/adrg/xdg"
 "github.com/spf13/cobra"
)

const desktopAgentAddress = "127.0.0.1:48176"
const desktopAgentMaxQueue = 16
const desktopAgentMaxHistory = 20
const desktopAgentHistoryFilename = "desktop-agent-history.json"

var openDesktopAgentPage = openDesktopAgentPageURL
var desktopAgentBaseURL = "http://" + desktopAgentAddress
var desktopAgentExecutable = os.Executable
var desktopAgentBackgroundStarter = startDesktopAgentBackgroundProcess
var desktopAgentReadyWaiter = waitForDesktopAgentReady

type desktopAgentTask struct {
 Action string   `json:"action"`
 Paths  []string `json:"paths"`
}

type desktopAgentTaskRecord struct {
 ID                int        `json:"id"`
 Action            string     `json:"action"`
 Paths             []string   `json:"paths"`
 State             string     `json:"state"`
 TransferState     string     `json:"transferState,omitempty"`
 TransferMessage   string     `json:"transferMessage,omitempty"`
 TransferMode      string     `json:"transferMode,omitempty"`
 TransferTarget    string     `json:"transferTarget,omitempty"`
 TransferArchive   bool       `json:"transferArchive,omitempty"`
 TransferArchiveName string   `json:"transferArchiveName,omitempty"`
 TransferItems     []string   `json:"transferItems,omitempty"`
 TransferCurrent   string     `json:"transferCurrent,omitempty"`
 TransferPercent   int        `json:"transferPercent,omitempty"`
 BytesDone         int64      `json:"bytesDone,omitempty"`
 BytesTotal        int64      `json:"bytesTotal,omitempty"`
 SavedFiles        []string   `json:"savedFiles,omitempty"`
 ChatState         string     `json:"chatState,omitempty"`
 ChatMessageCount  int        `json:"chatMessageCount,omitempty"`
 ChatDeviceCount   int        `json:"chatDeviceCount,omitempty"`
```

```
 ChatLastActivity    string     ‘json:"chatLastActivity,omitempty"‘
 PageURL             string     ‘json:"pageUrl,omitempty"‘
 Error               string     ‘json:"error,omitempty"‘
 StartedAt           time.Time  ‘json:"startedAt"‘
 FinishedAt          *time.Time ‘json:"finishedAt,omitempty"‘
}

type desktopAgentResponse struct {
 State          string                   ‘json:"state"‘
 Current        *desktopAgentTaskRecord  ‘json:"current,omitempty"‘
 Queued         int                      ‘json:"queued"‘
 History        []desktopAgentTaskRecord ‘json:"history,omitempty"‘
 LastError      string                   ‘json:"lastError,omitempty"‘
 Version        string                   ‘json:"version"‘
 AgentStartedAt time.Time                ‘json:"agentStartedAt"‘
}

type desktopAgentHistoryStore struct {
 History []desktopAgentTaskRecord ‘json:"history"‘
}

type desktopAgentNotifier func(title string, message string) error

type desktopAgent struct {
 mu          sync.Mutex
 baseFlags   application.Flags
 startedAt   time.Time
 busy        bool
 current     *desktopAgentTaskRecord
 queue       []desktopAgentTask
 history     []desktopAgentTaskRecord
 nextID      int
 activeStop  func(string)
 shutdown    func()
 lastError   string
 runner      func(desktopAgentTask) error
 notifier    desktopAgentNotifier
 historyPath string
 notified    map[int]map[string]bool
 revision    int64
 subscribers map[chan struct{}]struct{}
}

func newDesktopAgent(baseFlags application.Flags) *desktopAgent {
 agent := &desktopAgent{
  baseFlags:   baseFlags,
  startedAt:   time.Now(),
  notifier:    notifyDesktop,
  historyPath: defaultDesktopAgentHistoryPath(),
  notified:    map[int]map[string]bool{},
```

```
  subscribers: map[chan struct{}]struct{}{},
 }
 agent.runner = agent.runTask
 return agent
}

func defaultDesktopAgentHistoryPath() string {
 return filepath.Join(xdg.ConfigHome, application.New().Name, desktopAgentHistoryFilename)
}

func (agent *desktopAgent) routes() http.Handler {
 mux := http.NewServeMux()
 mux.HandleFunc("/", agent.handlePage)
 mux.HandleFunc("/health", agent.handleHealth)
 mux.HandleFunc("/status", agent.handleStatus)
 mux.HandleFunc("/events", agent.handleEvents)
 mux.HandleFunc("/settings", agent.handleSettings)
 mux.HandleFunc("/restart", agent.handleRestart)
 mux.HandleFunc("/tasks", agent.handleTasks)
 mux.HandleFunc("/tasks/", agent.handleTaskAction)
 mux.HandleFunc("/history", agent.handleHistory)
 mux.HandleFunc("/stop-current", agent.handleStopCurrent)
 mux.HandleFunc("/shutdown", agent.handleShutdown)
 return mux
}

func (agent *desktopAgent) handlePage(w http.ResponseWriter, r *http.Request) {
 if r.URL.Path != "/" {
  http.NotFound(w, r)
  return
 }
 if r.Method != http.MethodGet {
  http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
  return
 }
 status := agent.snapshot()
 w.Header().Set("Content-Type", "text/html; charset=utf-8")
 _ = desktopAgentPageTemplate.Execute(w, status)
}

func (agent *desktopAgent) handleHealth(w http.ResponseWriter, r *http.Request) {
 if handleDesktopAgentCORS(w, r, http.MethodGet) {
  w.WriteHeader(http.StatusNoContent)
  return
 }
 if rejectCrossOriginDesktopAgent(w, r) {
  return
 }
 if r.Method != http.MethodGet {
  http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
```

```
  return
 }
 w.WriteHeader(http.StatusNoContent)
}

func (agent *desktopAgent) handleStatus(w http.ResponseWriter, r *http.Request) {
 if handleDesktopAgentCORS(w, r, http.MethodGet) {
  w.WriteHeader(http.StatusNoContent)
  return
 }
 if rejectCrossOriginDesktopAgent(w, r) {
  return
 }
 if r.Method != http.MethodGet {
  http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
  return
 }
 agent.writeStatus(w)
}

func (agent *desktopAgent) handleEvents(w http.ResponseWriter, r *http.Request) {
 if handleDesktopAgentCORS(w, r, http.MethodGet) {
  w.WriteHeader(http.StatusNoContent)
  return
 }
 if rejectCrossOriginDesktopAgent(w, r) {
  return
 }
 if r.Method != http.MethodGet {
  http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
  return
 }
 flusher, ok := w.(http.Flusher)
 if !ok {
  http.Error(w, "streaming unsupported", http.StatusInternalServerError)
  return
 }
 w.Header().Set("Content-Type", "text/event-stream")
 w.Header().Set("Cache-Control", "no-cache")
 w.Header().Set("Connection", "keep-alive")
 lastRevision := int64(-1)
 send := func() bool {
  status, revision := agent.snapshotWithRevision()
  if revision == lastRevision {
   return true
  }
  lastRevision = revision
  if _, err := fmt.Fprint(w, "data: "); err != nil {
   return false
  }
```

```
  if err := json.NewEncoder(w).Encode(status); err != nil {
   return false
  }
  if _, err := fmt.Fprint(w, "\n"); err != nil {
   return false
  }
  flusher.Flush()
  return true
 }
 if !send() {
  return
 }
 events, unsubscribe := agent.subscribeEvents()
 defer unsubscribe()
 heartbeat := time.NewTicker(15 * time.Second)
 defer heartbeat.Stop()
 for {
  select {
  case <-r.Context().Done():
   return
  case <-events:
   if !send() {
    return
   }
  case <-heartbeat.C:
   if _, err := fmt.Fprint(w, ": keep-alive\n\n"); err != nil {
    return
   }
   flusher.Flush()
  }
 }
}

func (agent *desktopAgent) subscribeEvents() (<-chan struct{}, func()) {
 ch := make(chan struct{}, 1)
 agent.mu.Lock()
 if agent.subscribers == nil {
  agent.subscribers = map[chan struct{}]struct{}{}
 }
 agent.subscribers[ch] = struct{}{}
 agent.mu.Unlock()
 return ch, func() {
  agent.mu.Lock()
  delete(agent.subscribers, ch)
  close(ch)
  agent.mu.Unlock()
 }
}

func (agent *desktopAgent) touchLocked() {
```

```
 agent.revision++
 for ch := range agent.subscribers {
  select {
  case ch <- struct{}{}:
  default:
  }
 }
}

func (agent *desktopAgent) handleRestart(w http.ResponseWriter, r *http.Request) {
 if rejectCrossOriginDesktopAgent(w, r) {
  return
 }
 if r.Method != http.MethodPost {
  http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
  return
 }
 logFile, logPath, err := createDesktopAgentBackgroundLog()
 if err != nil {
  http.Error(w, fmt.Sprintf("desktop agent restart failed: %v", err), http.StatusInternalServerEr
  return
 }
 exe, err := desktopAgentExecutable()
 if err != nil {
  _ = logFile.Close()
  http.Error(w, fmt.Sprintf("desktop agent restart failed: %v", err), http.StatusInternalServerEr
  return
 }
 cmd := exec.Command(exe, "desktop", "agent-restart-helper")
 configureDesktopAgentBackgroundCommand(cmd)
 cmd.Stdout = logFile
 cmd.Stderr = logFile
 if err := cmd.Start(); err != nil {
  _ = logFile.Close()
  http.Error(w, fmt.Sprintf("desktop agent restart failed: %v", err), http.StatusInternalServerEr
  return
 }
 if err := cmd.Process.Release(); err != nil {
  _ = logFile.Close()
  http.Error(w, fmt.Sprintf("desktop agent restart failed: %v", err), http.StatusInternalServerEr
  return
 }
 agent.mu.Lock()
 agent.queue = nil
 if agent.busy {
  agent.finalizeActiveLocked("replaced")
 }
 shutdown := agent.shutdown
 agent.touchLocked()
 agent.mu.Unlock()
```

```
 w.WriteHeader(http.StatusAccepted)
 fmt.Fprintf(w, "Desktop agent restarting.\nLog: %s\n", logPath)
 go func() {
  defer logFile.Close()
  if shutdown != nil {
   shutdown()
  }
 }()
}

func (agent *desktopAgent) handleSettings(w http.ResponseWriter, r *http.Request) {
 if rejectCrossOriginDesktopAgent(w, r) {
  return
 }
 switch r.Method {
 case http.MethodGet:
  settings, err := agent.readSettings()
  if err != nil {
   http.Error(w, fmt.Sprintf("desktop agent settings unavailable: %v", err), http.StatusInternalS
   return
  }
  w.Header().Set("Content-Type", "application/json")
  _ = json.NewEncoder(w).Encode(settings)
 case http.MethodPost:
  var settings config.DesktopSettings
  if err := json.NewDecoder(r.Body).Decode(&settings); err != nil {
   http.Error(w, fmt.Sprintf("invalid settings: %v", err), http.StatusBadRequest)
   return
  }
  saved, err := agent.writeSettings(settings)
  if err != nil {
   http.Error(w, fmt.Sprintf("save desktop agent settings failed: %v", err), http.StatusBadReques
   return
  }
  w.Header().Set("Content-Type", "application/json")
  _ = json.NewEncoder(w).Encode(saved)
 default:
  http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
 }
}

func (agent *desktopAgent) settingsApp() application.App {
 settingsApp := application.New()
 settingsApp.Flags = agent.baseFlags
 return settingsApp
}

func (agent *desktopAgent) readSettings() (config.DesktopSettings, error) {
 return config.ReadDesktopSettings(agent.settingsApp())
```

```
}

func (agent *desktopAgent) writeSettings(settings config.DesktopSettings) (config.DesktopSettin
 return config.WriteDesktopSettings(agent.settingsApp(), settings)
}

func (agent *desktopAgent) handleTasks(w http.ResponseWriter, r *http.Request) {
 if rejectCrossOriginDesktopAgent(w, r) {
  return
 }
 if r.Method != http.MethodPost {
  http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
  return
 }
 var task desktopAgentTask
 if err := json.NewDecoder(r.Body).Decode(&task); err != nil {
  http.Error(w, fmt.Sprintf("invalid task: %v", err), http.StatusBadRequest)
  return
 }
 if err := validateDesktopAgentTask(task); err != nil {
  http.Error(w, err.Error(), http.StatusBadRequest)
  return
 }
 agent.mu.Lock()
 if len(agent.queue) >= desktopAgentMaxQueue {
  agent.mu.Unlock()
  http.Error(w, "desktop agent queue is full", http.StatusTooManyRequests)
  return
 }
 agent.queue = append(agent.queue, task)
 agent.lastError = ""
 if agent.busy {
  agent.replaceActiveLocked("replaced")
 }
 agent.startNextLocked()
 agent.touchLocked()
 agent.mu.Unlock()

 w.WriteHeader(http.StatusAccepted)
 agent.writeStatus(w)
}

func (agent *desktopAgent) handleTaskAction(w http.ResponseWriter, r *http.Request) {
 if handleDesktopAgentCORS(w, r, http.MethodPost) {
  w.WriteHeader(http.StatusNoContent)
  return
 }
 if rejectCrossOriginDesktopAgent(w, r) {
  return
 }
```

```
 if r.Method != http.MethodPost {
  http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
  return
 }
 taskID, action, ok := parseDesktopAgentTaskActionPath(r.URL.Path)
 if !ok || action != "repeat" {
  http.NotFound(w, r)
  return
 }
 if err := agent.repeatTask(taskID); err != nil {
  http.Error(w, err.Error(), http.StatusConflict)
  return
 }
 w.WriteHeader(http.StatusAccepted)
 agent.writeStatus(w)
}

func parseDesktopAgentTaskActionPath(path string) (int, string, bool) {
 parts := strings.Split(strings.Trim(path, "/"), "/")
 if len(parts) != 3 || parts[0] != "tasks" {
  return 0, "", false
 }
 id, err := strconv.Atoi(parts[1])
 if err != nil || id <= 0 {
  return 0, "", false
 }
 return id, parts[2], true
}

func handleDesktopAgentCORS(w http.ResponseWriter, r *http.Request, methods ...string) bool {
 origin := r.Header.Get("Origin")
 if origin != "" && trustedDesktopAgentOrigin(origin, r.Host) {
  w.Header().Set("Access-Control-Allow-Origin", origin)
  w.Header().Set("Vary", "Origin")
  w.Header().Set("Access-Control-Allow-Headers", "Content-Type")
  if len(methods) > 0 {
   allowed := append([]string(nil), methods...)
   allowed = append(allowed, http.MethodOptions)
   w.Header().Set("Access-Control-Allow-Methods", strings.Join(allowed, ", "))
  }
 }
 return r.Method == http.MethodOptions
}

func rejectCrossOriginDesktopAgent(w http.ResponseWriter, r *http.Request) bool {
 origin := r.Header.Get("Origin")
 if origin == "" {
  return false
 }
 if trustedDesktopAgentOrigin(origin, r.Host) {
```

```
  return false
 }
 http.Error(w, "forbidden", http.StatusForbidden)
 return true
}

func trustedDesktopAgentOrigin(origin string, requestHost string) bool {
 parsed, err := url.Parse(origin)
 if err != nil {
  return false
 }
 if trustedDesktopAgentWailsOrigin(parsed) {
  return true
 }
 if strings.EqualFold(parsed.Host, requestHost) {
  return true
 }
 return trustedDesktopAgentLocalHost(parsed.Hostname())
}

func trustedDesktopAgentWailsOrigin(parsed *url.URL) bool {
 host := strings.ToLower(parsed.Hostname())
 return parsed.Scheme == "wails" || host == "wails.localhost"
}

func trustedDesktopAgentLocalHost(host string) bool {
 host = strings.Trim(strings.ToLower(host), "[]")
 if host == "localhost" {
  return true
 }
 ip := net.ParseIP(host)
 return ip != nil && (ip.IsLoopback() || ip.IsPrivate() || ip.IsLinkLocalUnicast())
}

func (agent *desktopAgent) handleHistory(w http.ResponseWriter, r *http.Request) {
 if rejectCrossOriginDesktopAgent(w, r) {
  return
 }
 if r.Method != http.MethodDelete {
  http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
  return
 }
 if err := agent.clearHistory(); err != nil {
  http.Error(w, fmt.Sprintf("clear desktop agent history failed: %v", err), http.StatusInternalSe
  return
 }
 w.WriteHeader(http.StatusNoContent)
}

func (agent *desktopAgent) handleStopCurrent(w http.ResponseWriter, r *http.Request) {
```

```
 if rejectCrossOriginDesktopAgent(w, r) {
  return
 }
 if r.Method != http.MethodPost {
  http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
  return
 }
 if !agent.stopCurrent("stopped") {
  http.Error(w, "desktop agent has no active task", http.StatusConflict)
  return
 }
 if strings.Contains(r.Header.Get("Accept"), "text/html") {
  http.Redirect(w, r, "/", http.StatusSeeOther)
  return
 }
 w.WriteHeader(http.StatusAccepted)
 fmt.Fprintln(w, "Current desktop agent task stopped.")
}

func (agent *desktopAgent) handleShutdown(w http.ResponseWriter, r *http.Request) {
 if rejectCrossOriginDesktopAgent(w, r) {
  return
 }
 if r.Method != http.MethodPost {
  http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
  return
 }
 agent.mu.Lock()
 agent.queue = nil
 if agent.busy {
  agent.finalizeActiveLocked("replaced")
 }
 shutdown := agent.shutdown
 agent.touchLocked()
 agent.mu.Unlock()

 w.WriteHeader(http.StatusAccepted)
 fmt.Fprintln(w, "Desktop agent stopping.")
 if shutdown != nil {
  go shutdown()
 }
}

func (agent *desktopAgent) execute(task desktopAgentTask, id int) {
 err := agent.runner(task)
 agent.mu.Lock()
 defer agent.mu.Unlock()
 if agent.current != nil && agent.current.ID == id {
  finishedAt := time.Now()
  agent.current.FinishedAt = &finishedAt
```

```
  if agent.current.State == "running" {
   if err != nil {
    agent.current.State = "failed"
    agent.current.Error = err.Error()
    agent.lastError = err.Error()
   } else if isTerminalDesktopTransferState(agent.current.TransferState) {
    agent.current.State = agent.current.TransferState
   } else {
    agent.current.State = "completed"
   }
  }
  agent.addHistoryLocked(*agent.current)
  agent.notifyRecordLocked(*agent.current)
  delete(agent.notified, agent.current.ID)
  agent.busy = false
  agent.current = nil
  agent.activeStop = nil
  agent.startNextLocked()
  agent.touchLocked()
 }
}

func (agent *desktopAgent) stopCurrent(state string) bool {
 agent.mu.Lock()
 defer agent.mu.Unlock()
 if !agent.busy {
  return false
 }
 agent.replaceActiveLocked(state)
 agent.touchLocked()
 return true
}

func (agent *desktopAgent) repeatTask(id int) error {
 agent.mu.Lock()
 defer agent.mu.Unlock()
 if len(agent.queue) >= desktopAgentMaxQueue {
  return fmt.Errorf("desktop agent queue is full")
 }
 var repeated *desktopAgentTask
 if agent.current != nil && agent.current.ID == id {
  task := desktopAgentTask{Action: agent.current.Action, Paths: append([]string(nil), agent.curre
  repeated = &task
 }
 if repeated == nil {
  for _, record := range agent.history {
   if record.ID == id {
```

```
   }
  }
 }
 if repeated == nil {
  return fmt.Errorf("desktop agent task #%d was not found", id)
 }
 if err := validateDesktopAgentTask(*repeated); err != nil {
  return err
 }
 agent.queue = append(agent.queue, *repeated)
 agent.lastError = ""
 if agent.busy {
  agent.replaceActiveLocked("replaced")
 }
 agent.startNextLocked()
 agent.touchLocked()
 return nil
}

func isTerminalDesktopTransferState(state string) bool {
 return state == "completed" || state == "stopped" || state == "failed"
}

func isTerminalDesktopChatState(state string) bool {
 return state == "ended" || state == "stopped" || state == "failed" || state == "replaced"
}

func desktopTaskStateForChatState(state string) string {
 switch state {
 case "ended":
  return "completed"
 case "stopped", "failed", "replaced":
  return state
 default:
  return "running"
 }
}

func (agent *desktopAgent) replaceActiveLocked(state string) {
 if agent.current != nil && agent.current.State == "running" {
  agent.current.State = state
  finishedAt := time.Now()
  agent.current.FinishedAt = &finishedAt
 }
 if agent.activeStop == nil {
  return
 }
 stop := agent.activeStop
 go stop(state)
}
```

```
func (agent *desktopAgent) finalizeActiveLocked(state string) {
 agent.replaceActiveLocked(state)
 if agent.current == nil {
  return
 }
 record := *agent.current
 agent.addHistoryLocked(record)
 agent.notifyRecordLocked(record)
 delete(agent.notified, record.ID)
 agent.busy = false
 agent.current = nil
 agent.activeStop = nil
}

func (agent *desktopAgent) startNextLocked() {
 if agent.busy || len(agent.queue) == 0 {
  return
 }
 task := agent.queue[0]
 agent.queue = agent.queue[1:]
 agent.nextID++
 record := desktopAgentTaskRecord{
  ID:        agent.nextID,
  Action:    task.Action,
  Paths:     append([]string(nil), task.Paths...),
  State:     "running",
  StartedAt: time.Now(),
 }
 agent.busy = true
 agent.current = &record
 agent.notifyRecordLocked(record)
 go agent.execute(task, record.ID)
}

func (agent *desktopAgent) currentTaskID() int {
 agent.mu.Lock()
 defer agent.mu.Unlock()
 if agent.current == nil {
  return 0
 }
 return agent.current.ID
}

func (agent *desktopAgent) observeTransferStatus(taskID int, status server.TransferStatusSnapshot
 agent.mu.Lock()
 defer agent.mu.Unlock()
 if agent.current == nil || agent.current.ID != taskID {
  return
 }
```

```
 agent.current.TransferState = status.State
 agent.current.TransferMessage = status.Message
 agent.current.TransferMode = status.Mode
 agent.current.TransferTarget = status.Target
 agent.current.TransferArchive = status.Archive
 agent.current.TransferArchiveName = status.ArchiveName
 agent.current.TransferItems = append([]string(nil), status.Items...)
 agent.current.TransferCurrent = status.Current
 agent.current.TransferPercent = status.Percent
 agent.current.BytesDone = status.BytesDone
 agent.current.BytesTotal = status.BytesTotal
 agent.current.SavedFiles = append([]string(nil), status.SavedFiles...)
 if isTerminalDesktopTransferState(status.State) && agent.current.State == "running" {
  agent.current.State = status.State
  finishedAt := time.Now()
  agent.current.FinishedAt = &finishedAt
 }
 agent.notifyTransferStatusLocked(*agent.current)
 if isTerminalDesktopTransferState(agent.current.State) {
  record := *agent.current
  agent.addHistoryLocked(record)
  agent.notifyRecordLocked(record)
  delete(agent.notified, record.ID)
  agent.busy = false
  agent.current = nil
  agent.activeStop = nil
  agent.startNextLocked()
  agent.touchLocked()
  return
 }
 agent.touchLocked()
}

func (agent *desktopAgent) observeChatStatus(taskID int, status server.ChatStatusSnapshot) {
 agent.mu.Lock()
 defer agent.mu.Unlock()
 if agent.current == nil || agent.current.ID != taskID {
  return
 }
 agent.current.ChatState = status.State
 agent.current.ChatMessageCount = status.MessageCount
 agent.current.ChatDeviceCount = status.DeviceCount
 if !status.LastActivity.IsZero() {
  agent.current.ChatLastActivity = status.LastActivity.Format(time.RFC3339)
 }
 if isTerminalDesktopChatState(status.State) && agent.current.State == "running" {
  agent.current.State = desktopTaskStateForChatState(status.State)
  finishedAt := time.Now()
  agent.current.FinishedAt = &finishedAt
  record := *agent.current
```

```
  agent.addHistoryLocked(record)
  agent.notifyRecordLocked(record)
  delete(agent.notified, record.ID)
  agent.busy = false
  agent.current = nil
  agent.activeStop = nil
  agent.startNextLocked()
  agent.touchLocked()
  return
 }
 agent.touchLocked()
}

func (agent *desktopAgent) notifyRecordLocked(record desktopAgentTaskRecord) {
 if agent.notifier == nil {
  return
 }
 title, message := desktopAgentNotification(record)
 if title == "" || message == "" {
  return
 }
 _ = agent.notifier(title, message)
}

func (agent *desktopAgent) notifyTransferStatusLocked(record desktopAgentTaskRecord) {
 if agent.notifier == nil {
  return
 }
 key := record.TransferState
 switch key {
 case "transferring", "completed", "stopped", "failed":
 default:
  return
 }
 if agent.notified[record.ID] == nil {
  agent.notified[record.ID] = map[string]bool{}
 }
 if agent.notified[record.ID][key] {
  return
 }
 title, message := desktopAgentTransferNotification(record)
 if title == "" || message == "" {
  return
 }
 agent.notified[record.ID][key] = true
 _ = agent.notifier(title, message)
}

func desktopAgentNotification(record desktopAgentTaskRecord) (string, string) {
 if record.Action == "chat" {
```

```
 return "", ""
 }
 action := desktopAgentActionLabel(record.Action)
 target := desktopAgentPathsSummary(record.Paths)
 switch record.State {
 case "running":
  return "eqrcp transfer ready", fmt.Sprintf("%s ready: %s", action, target)
 case "completed":
  if record.TransferState == "completed" {
   return "", ""
  }
  return "eqrcp transfer completed", fmt.Sprintf("%s completed: %s", action, target)
 case "failed":
  if record.TransferState == "failed" {
   return "", ""
  }
  if record.Error != "" {
   return "eqrcp transfer failed", fmt.Sprintf("%s failed: %s", action, record.Error)
  }
  return "eqrcp transfer failed", fmt.Sprintf("%s failed: %s", action, target)
 case "stopped":
  if record.TransferState == "stopped" {
   return "", ""
  }
  return "eqrcp transfer stopped", fmt.Sprintf("%s stopped: %s", action, target)
 case "replaced":
  return "eqrcp transfer replaced", fmt.Sprintf("%s replaced by a newer task: %s", action, target)
 default:
  return "", ""
 }
}

func desktopAgentTransferNotification(record desktopAgentTaskRecord) (string, string) {
 action := desktopAgentActionLabel(record.Action)
 target := desktopAgentPathsSummary(record.Paths)
 if record.TransferCurrent != "" {
  target = record.TransferCurrent
 }
 switch record.TransferState {
 case "transferring":
  return "eqrcp transfer started", fmt.Sprintf("%s started: %s", action, target)
 case "completed":
  if len(record.SavedFiles) == 1 {
```

```
  if (this._stream) this._stream.emit('close')
}

Extract.prototype._parse = function (size, onparse) {
  if (this._destroyed) return
  this._offset += size
  this._missing = size
  if (onparse === this._onheader) this._partial = false
  this._onparse = onparse
}

Extract.prototype._continue = function () {
  if (this._destroyed) return
  var cb = this._cb
  this._cb = noop
  if (this._overflow) this._write(this._overflow, undefined, cb)
  else cb()
}

Extract.prototype._write = function (data, enc, cb) {
  if (this._destroyed) return

  var s = this._stream
  var b = this._buffer
  var missing = this._missing
  if (data.length) this._partial = true

  // we do not reach end-of-chunk now. just forward it

  if (data.length < missing) {
    this._missing -= data.length
    this._overflow = null
    if (s) return s.write(data, cb)
    b.append(data)
    return cb()
  }

  // end-of-chunk. the parser should call cb.

  this._cb = cb
  this._missing = 0

  var overflow = null
  if (data.length > missing) {
    overflow = data.slice(missing)
    data = data.slice(0, missing)
  }

  if (s) s.end(data)
  else b.append(data)
```

```
  this._overflow = overflow
  this._onparse()
}

Extract.prototype._final = function (cb) {
  if (this._partial) return this.destroy(new Error('Unexpected end of data'))
  cb()
}

module.exports = Extract
var alloc = Buffer.alloc

var ZEROS = '0000000000000000000'
var SEVENS = '7777777777777777777'
var ZERO_OFFSET = '0'.charCodeAt(0)
var USTAR_MAGIC = Buffer.from('ustar\x00', 'binary')
var USTAR_VER = Buffer.from('00', 'binary')
var GNU_MAGIC = Buffer.from('ustar\x20', 'binary')
var GNU_VER = Buffer.from('\x20\x00', 'binary')
var MASK = parseInt('7777', 8)
var MAGIC_OFFSET = 257
var VERSION_OFFSET = 263

var clamp = function (index, len, defaultValue) {
  if (typeof index !== 'number') return defaultValue
  index = ~~index // Coerce to integer.
  if (index >= len) return len
  if (index >= 0) return index
  index += len
  if (index >= 0) return index
  return 0
}

var toType = function (flag) {
  switch (flag) {
    case 0:
      return 'file'
    case 1:
      return 'link'
    case 2:
      return 'symlink'
    case 3:
      return 'character-device'
    case 4:
      return 'block-device'
    case 5:
      return 'directory'
    case 6:
      return 'fifo'
```

```
    case 7:
      return 'contiguous-file'
    case 72:
      return 'pax-header'
    case 55:
      return 'pax-global-header'
    case 27:
      return 'gnu-long-link-path'
    case 28:
    case 30:
      return 'gnu-long-path'
  }

  return null
}

var toTypeflag = function (flag) {
  switch (flag) {
    case 'file':
      return 0
    case 'link':
      return 1
    case 'symlink':
      return 2
    case 'character-device':
      return 3
    case 'block-device':
      return 4
    case 'directory':
      return 5
    case 'fifo':
      return 6
    case 'contiguous-file':
      return 7
    case 'pax-header':
      return 72
  }

  return 0
}

var indexOf = function (block, num, offset, end) {
  for (; offset < end; offset++) {
    if (block[offset] === num) return offset
  }
  return end
}

var cksum = function (block) {
  var sum = 8 * 32
```

```
  for (var i = 0; i < 148; i++) sum += block[i]
  for (var j = 156; j < 512; j++) sum += block[j]
  return sum
}

var encodeOct = function (val, n) {
  val = val.toString(8)
  if (val.length > n) return SEVENS.slice(0, n) + ' '
  else return ZEROS.slice(0, n - val.length) + val + ' '
}

/* Copied from the node-tar repo and modified to meet
 * tar-stream coding standard.
 *
 * Source: https://github.com/npm/node-tar/blob/51b6627a1f357d2eb433e7378e5f05e83b7aa6cd/lib/header.
 */
function parse256 (buf) {
  // first byte MUST be either 80 or FF
  // 80 for positive, FF for 2's comp
  var positive
  if (buf[0] === 0x80) positive = true
  else if (buf[0] === 0xFF) positive = false
  else return null

  // build up a base-256 tuple from the least sig to the highest
  var tuple = []
  for (var i = buf.length - 1; i > 0; i--) {
    var byte = buf[i]
    if (positive) tuple.push(byte)
    else tuple.push(0xFF - byte)
  }

  var sum = 0
  var l = tuple.length
  for (i = 0; i < l; i++) {
    sum += tuple[i] * Math.pow(256, i)
  }

  return positive ? sum : -1 * sum
}

var decodeOct = function (val, offset, length) {
  val = val.slice(offset, offset + length)
  offset = 0

  // If prefixed with 0x80 then parse as a base-256 integer
  if (val[offset] & 0x80) {
    return parse256(val)
  } else {
    // Older versions of tar can prefix with spaces
```

```
    while (offset < val.length && val[offset] === 32) offset++
    var end = clamp(indexOf(val, 32, offset, val.length), val.length, val.length)
    while (offset < end && val[offset] === 0) offset++
    if (end === offset) return 0
    return parseInt(val.slice(offset, end).toString(), 8)
  }
}

var decodeStr = function (val, offset, length, encoding) {
  return val.slice(offset, indexOf(val, 0, offset, offset + length)).toString(encoding)
}

var addLength = function (str) {
  var len = Buffer.byteLength(str)
  var digits = Math.floor(Math.log(len) / Math.log(10)) + 1
  if (len + digits >= Math.pow(10, digits)) digits++

  return (len + digits) + str
}

exports.decodeLongPath = function (buf, encoding) {
  return decodeStr(buf, 0, buf.length, encoding)
}

exports.encodePax = function (opts) { // TODO: encode more stuff in pax
  var result = ''
  if (opts.name) result += addLength(' path=' + opts.name + '\n')
  if (opts.linkname) result += addLength(' linkpath=' + opts.linkname + '\n')
  var pax = opts.pax
  if (pax) {
    for (var key in pax) {
      result += addLength(' ' + key + '=' + pax[key] + '\n')
    }
  }
  return Buffer.from(result)
}

exports.decodePax = function (buf) {
  var result = {}

  while (buf.length) {
    var i = 0
    while (i < buf.length && buf[i] !== 32) i++
    var len = parseInt(buf.slice(0, i).toString(), 10)
    if (!len) return result

    var b = buf.slice(i + 1, len - 1).toString()
    var keyIndex = b.indexOf('=')
    if (keyIndex === -1) return result
    result[b.slice(0, keyIndex)] = b.slice(keyIndex + 1)
```

```
    buf = buf.slice(len)
  }

  return result
}

exports.encode = function (opts) {
  var buf = alloc(512)
  var name = opts.name
  var prefix = ''

  if (opts.typeflag === 5 && name[name.length - 1] !== '/') name += '/'
  if (Buffer.byteLength(name) !== name.length) return null // utf-8

  while (Buffer.byteLength(name) > 100) {
    var i = name.indexOf('/')
    if (i === -1) return null
    prefix += prefix ? '/' + name.slice(0, i) : name.slice(0, i)
    name = name.slice(i + 1)
  }

  if (Buffer.byteLength(name) > 100 || Buffer.byteLength(prefix) > 155) return null
  if (opts.linkname && Buffer.byteLength(opts.linkname) > 100) return null

  buf.write(name)
  buf.write(encodeOct(opts.mode & MASK, 6), 100)
  buf.write(encodeOct(opts.uid, 6), 108)
  buf.write(encodeOct(opts.gid, 6), 116)
  buf.write(encodeOct(opts.size, 11), 124)
  buf.write(encodeOct((opts.mtime.getTime() / 1000) | 0, 11), 136)

  buf[156] = ZERO_OFFSET + toTypeflag(opts.type)

  if (opts.linkname) buf.write(opts.linkname, 157)

  USTAR_MAGIC.copy(buf, MAGIC_OFFSET)
  USTAR_VER.copy(buf, VERSION_OFFSET)
  if (opts.uname) buf.write(opts.uname, 265)
  if (opts.gname) buf.write(opts.gname, 297)
  buf.write(encodeOct(opts.devmajor || 0, 6), 329)
  buf.write(encodeOct(opts.devminor || 0, 6), 337)

  if (prefix) buf.write(prefix, 345)

  buf.write(encodeOct(cksum(buf), 6), 148)

  return buf
}
```

```
exports.decode = function (buf, filenameEncoding, allowUnknownFormat) {
  var typeflag = buf[156] === 0 ? 0 : buf[156] - ZERO_OFFSET

  var name = decodeStr(buf, 0, 100, filenameEncoding)
  var mode = decodeOct(buf, 100, 8)
  var uid = decodeOct(buf, 108, 8)
  var gid = decodeOct(buf, 116, 8)
  var size = decodeOct(buf, 124, 12)
  var mtime = decodeOct(buf, 136, 12)
  var type = toType(typeflag)
  var linkname = buf[157] === 0 ? null : decodeStr(buf, 157, 100, filenameEncoding)
  var uname = decodeStr(buf, 265, 32)
  var gname = decodeStr(buf, 297, 32)
  var devmajor = decodeOct(buf, 329, 8)
  var devminor = decodeOct(buf, 337, 8)

  var c = cksum(buf)

  // checksum is still initial value if header was null.
  if (c === 8 * 32) return null

  // valid checksum
  if (c !== decodeOct(buf, 148, 8)) throw new Error('Invalid tar header. Maybe the tar is corru

  if (USTAR_MAGIC.compare(buf, MAGIC_OFFSET, MAGIC_OFFSET + 6) === 0) {
    // ustar (posix) format.
    // prepend prefix, if present.
    if (buf[345]) name = decodeStr(buf, 345, 155, filenameEncoding) + '/' + name
  } else if (GNU_MAGIC.compare(buf, MAGIC_OFFSET, MAGIC_OFFSET + 6) === 0 &&
             GNU_VER.compare(buf, VERSION_OFFSET, VERSION_OFFSET + 2) === 0) {
    // 'gnu'/'oldgnu' format. Similar to ustar, but has support for incremental and
    // multi-volume tarballs.
  } else {
    if (!allowUnknownFormat) {
      throw new Error('Invalid tar header: unknown format.')
    }
  }

  // to support old tar versions that use trailing / to indicate dirs
  if (typeflag === 0 && name && name[name.length - 1] === '/') typeflag = 5

  return {
    name,
    mode,
    uid,
    gid,
    size,
    mtime: new Date(1000 * mtime),
    type,
    linkname,
```

```
    uname,
    gname,
    devmajor,
    devminor
  }
}
exports.extract = require('./extract')
exports.pack = require('./pack')
var constants = require('fs-constants')
var eos = require('end-of-stream')
var inherits = require('inherits')
var alloc = Buffer.alloc

var Readable = require('readable-stream').Readable
var Writable = require('readable-stream').Writable
var StringDecoder = require('string_decoder').StringDecoder

var headers = require('./headers')

var DMODE = parseInt('755', 8)
var FMODE = parseInt('644', 8)

var END_OF_TAR = alloc(1024)

var noop = function () {}

var overflow = function (self, size) {
  size &= 511
  if (size) self.push(END_OF_TAR.slice(0, 512 - size))
}

function modeToType (mode) {
  switch (mode & constants.S_IFMT) {
    case constants.S_IFBLK: return 'block-device'
    case constants.S_IFCHR: return 'character-device'
    case constants.S_IFDIR: return 'directory'
    case constants.S_IFIFO: return 'fifo'
    case constants.S_IFLNK: return 'symlink'
  }

  return 'file'
}

var Sink = function (to) {
  Writable.call(this)
  this.written = 0
  this._to = to
  this._destroyed = false
}
```

```
inherits(Sink, Writable)

Sink.prototype._write = function (data, enc, cb) {
  this.written += data.length
  if (this._to.push(data)) return cb()
  this._to._drain = cb
}

Sink.prototype.destroy = function () {
  if (this._destroyed) return
  this._destroyed = true
  this.emit('close')
}

var LinkSink = function () {
  Writable.call(this)
  this.linkname = ''
  this._decoder = new StringDecoder('utf-8')
  this._destroyed = false
}

inherits(LinkSink, Writable)

LinkSink.prototype._write = function (data, enc, cb) {
  this.linkname += this._decoder.write(data)
  cb()
}

LinkSink.prototype.destroy = function () {
  if (this._destroyed) return
  this._destroyed = true
  this.emit('close')
}

var Void = function () {
  Writable.call(this)
  this._destroyed = false
}

inherits(Void, Writable)

Void.prototype._write = function (data, enc, cb) {
  cb(new Error('No body allowed for this entry'))
}

Void.prototype.destroy = function () {
  if (this._destroyed) return
  this._destroyed = true
  this.emit('close')
}
```

```
var Pack = function (opts) {
  if (!(this instanceof Pack)) return new Pack(opts)
  Readable.call(this, opts)

  this._drain = noop
  this._finalized = false
  this._finalizing = false
  this._destroyed = false
  this._stream = null
}

inherits(Pack, Readable)

Pack.prototype.entry = function (header, buffer, callback) {
  if (this._stream) throw new Error('already piping an entry')
  if (this._finalized || this._destroyed) return

  if (typeof buffer === 'function') {
    callback = buffer
    buffer = null
  }

  if (!callback) callback = noop

  var self = this

  if (!header.size || header.type === 'symlink') header.size = 0
  if (!header.type) header.type = modeToType(header.mode)
  if (!header.mode) header.mode = header.type === 'directory' ? DMODE : FMODE
  if (!header.uid) header.uid = 0
  if (!header.gid) header.gid = 0
  if (!header.mtime) header.mtime = new Date()

  if (typeof buffer === 'string') buffer = Buffer.from(buffer)
  if (Buffer.isBuffer(buffer)) {
    header.size = buffer.length
    this._encode(header)
    var ok = this.push(buffer)
    overflow(self, header.size)
    if (ok) process.nextTick(callback)
    else this._drain = callback
    return new Void()
  }

  if (header.type === 'symlink' && !header.linkname) {
    var linkSink = new LinkSink()
    eos(linkSink, function (err) {
      if (err) { // stream was closed
        self.destroy()
```

```
        return callback(err)
      }

      header.linkname = linkSink.linkname
      self._encode(header)
      callback()
    })

    return linkSink
  }

  this._encode(header)

  if (header.type !== 'file' && header.type !== 'contiguous-file') {
    process.nextTick(callback)
    return new Void()
  }

  var sink = new Sink(this)

  this._stream = sink

  eos(sink, function (err) {
    self._stream = null

    if (err) { // stream was closed
      self.destroy()
      return callback(err)
    }

    if (sink.written !== header.size) { // corrupting tar
      self.destroy()
      return callback(new Error('size mismatch'))
    }

    overflow(self, header.size)
    if (self._finalizing) self.finalize()
    callback()
  })

  return sink
}

Pack.prototype.finalize = function () {
  if (this._stream) {
    this._finalizing = true
    return
  }

  if (this._finalized) return
```

```
  this._finalized = true
  this.push(END_OF_TAR)
  this.push(null)
}

Pack.prototype.destroy = function (err) {
  if (this._destroyed) return
  this._destroyed = true

  if (err) this.emit('error', err)
  this.emit('close')
  if (this._stream && this._stream.destroy) this._stream.destroy()
}

Pack.prototype._encode = function (header) {
  if (!header.pax) {
    var buf = headers.encode(header)
    if (buf) {
      this.push(buf)
      return
    }
  }
  this._encodePax(header)
}

Pack.prototype._encodePax = function (header) {
  var paxHeader = headers.encodePax({
    name: header.name,
    linkname: header.linkname,
    pax: header.pax
  })

  var newHeader = {
    name: 'PaxHeader',
    mode: header.mode,
    uid: header.uid,
    gid: header.gid,
    size: paxHeader.length,
    mtime: header.mtime,
    type: 'pax-header',
    linkname: header.linkname && 'PaxHeader',
    uname: header.uname,
    gname: header.gname,
    devmajor: header.devmajor,
    devminor: header.devminor
  }

  this.push(headers.encode(newHeader))
  this.push(paxHeader)
  overflow(this, paxHeader.length)
```

```
  newHeader.size = header.size
  newHeader.type = header.type
  this.push(headers.encode(newHeader))
}

Pack.prototype._read = function (n) {
  var drain = this._drain
  this._drain = noop
  drain()
}

module.exports = Pack
const tar = require('tar-stream')
const fs = require('fs')
const path = require('path')
const pipeline = require('pump') // eequire('stream').pipeline

fs.createReadStream('test.tar')
  .pipe(tar.extract())
  .on('entry', function (header, stream, done) {
    console.log(header.name)
    pipeline(stream, fs.createWriteStream(path.join('/tmp', header.name)), done)
  })
'use strict'

var net = require('net')
  , tls = require('tls')
  , http = require('http')
  , https = require('https')
  , events = require('events')
  , assert = require('assert')
  , util = require('util')
  , Buffer = require('safe-buffer').Buffer
  ;

exports.httpOverHttp = httpOverHttp
exports.httpsOverHttp = httpsOverHttp
exports.httpOverHttps = httpOverHttps
exports.httpsOverHttps = httpsOverHttps


function httpOverHttp(options) {
  var agent = new TunnelingAgent(options)
  agent.request = http.request
  return agent
}

function httpsOverHttp(options) {
  var agent = new TunnelingAgent(options)
```

```
  agent.request = http.request
  agent.createSocket = createSecureSocket
  agent.defaultPort = 443
  return agent
}

function httpOverHttps(options) {
  var agent = new TunnelingAgent(options)
  agent.request = https.request
  return agent
}

function httpsOverHttps(options) {
  var agent = new TunnelingAgent(options)
  agent.request = https.request
  agent.createSocket = createSecureSocket
  agent.defaultPort = 443
  return agent
}


function TunnelingAgent(options) {
  var self = this
  self.options = options || {}
  self.proxyOptions = self.options.proxy || {}
  self.maxSockets = self.options.maxSockets || http.Agent.defaultMaxSockets
  self.requests = []
  self.sockets = []

  self.on('free', function onFree(socket, host, port) {
    for (var i = 0, len = self.requests.length; i < len; ++i) {
      var pending = self.requests[i]
      if (pending.host === host && pending.port === port) {
        // Detect the request to connect same origin server,
        // reuse the connection.
        self.requests.splice(i, 1)
        pending.request.onSocket(socket)
        return
      }
    }
    socket.destroy()
    self.removeSocket(socket)
  })
}
util.inherits(TunnelingAgent, events.EventEmitter)

TunnelingAgent.prototype.addRequest = function addRequest(req, options) {
  var self = this

   // Legacy API: addRequest(req, host, port, path)
```

```
  if (typeof options === 'string') {
    options = {
      host: options,
      port: arguments[2],
      path: arguments[3]
    };
  }

  if (self.sockets.length >= this.maxSockets) {
    // We are over limit so we'll add it to the queue.
    self.requests.push({host: options.host, port: options.port, request: req})
    return
  }

  // If we are under maxSockets create a new one.
  self.createConnection({host: options.host, port: options.port, request: req})
}

TunnelingAgent.prototype.createConnection = function createConnection(pending) {
  var self = this

  self.createSocket(pending, function(socket) {
    socket.on('free', onFree)
    socket.on('close', onCloseOrRemove)
    socket.on('agentRemove', onCloseOrRemove)
    pending.request.onSocket(socket)

    function onFree() {
      self.emit('free', socket, pending.host, pending.port)
    }

    function onCloseOrRemove(err) {
      self.removeSocket(socket)
      socket.removeListener('free', onFree)
      socket.removeListener('close', onCloseOrRemove)
      socket.removeListener('agentRemove', onCloseOrRemove)
    }
  })
}

TunnelingAgent.prototype.createSocket = function createSocket(options, cb) {
  var self = this
  var placeholder = {}
  self.sockets.push(placeholder)

  var connectOptions = mergeOptions({}, self.proxyOptions,
    { method: 'CONNECT'
    , path: options.host + ':' + options.port
    , agent: false
    }
```

```
  )
  if (connectOptions.proxyAuth) {
    connectOptions.headers = connectOptions.headers || {}
    connectOptions.headers['Proxy-Authorization'] = 'Basic ' +
        Buffer.from(connectOptions.proxyAuth).toString('base64')
  }

  debug('making CONNECT request')
  var connectReq = self.request(connectOptions)
  connectReq.useChunkedEncodingByDefault = false // for v0.6
  connectReq.once('response', onResponse) // for v0.6
  connectReq.once('upgrade', onUpgrade)   // for v0.6
  connectReq.once('connect', onConnect)   // for v0.7 or later
  connectReq.once('error', onError)
  connectReq.end()

  function onResponse(res) {
    // Very hacky. This is necessary to avoid http-parser leaks.
    res.upgrade = true
  }

  function onUpgrade(res, socket, head) {
    // Hacky.
    process.nextTick(function() {
      onConnect(res, socket, head)
    })
  }

  function onConnect(res, socket, head) {
    connectReq.removeAllListeners()
    socket.removeAllListeners()

    if (res.statusCode === 200) {
      assert.equal(head.length, 0)
      debug('tunneling connection has established')
      self.sockets[self.sockets.indexOf(placeholder)] = socket
      cb(socket)
    } else {
      debug('tunneling socket could not be established, statusCode=%d', res.statusCode)
      var error = new Error('tunneling socket could not be established, ' + 'statusCode=' + res.sta
      error.code = 'ECONNRESET'
      options.request.emit('error', error)
      self.removeSocket(placeholder)
    }
  }

  function onError(cause) {
    connectReq.removeAllListeners()

    debug('tunneling socket could not be established, cause=%s\n', cause.message, cause.stack)
```

```
}


var debug
if (process.env.NODE_DEBUG && /\btunnel\b/.test(process.env.NODE_DEBUG)) {
  debug = function() {
    var args = Array.prototype.slice.call(arguments)
    if (typeof args[0] === 'string') {
      args[0] = 'TUNNEL: ' + args[0]
    } else {
      args.unshift('TUNNEL:')
    }
    console.error.apply(console, args)
  }
} else {
  debug = function() {}
}
exports.debug = debug // for test

/**
 * Module exports.
 */

module.exports = deprecate;

/**
 * Mark that a method should not be used.
 * Returns a modified function which warns once by default.
 *
 * If `localStorage.noDeprecation = true` is set, then it is a no-op.
 *
 * If `localStorage.throwDeprecation = true` is set, then deprecated functions
 * will throw an Error when invoked.
 *
 * If `localStorage.traceDeprecation = true` is set, then deprecated functions
 * will invoke `console.trace()` instead of `console.error()`.
 *
 * @param {Function} fn - the function to deprecate
 * @param {String} msg - the string to print to the console when `fn` is invoked
 * @returns {Function} a new "deprecated" version of `fn`
 * @api public
 */

function deprecate (fn, msg) {
  if (config('noDeprecation')) {
    return fn;
  }

  var warned = false;
  function deprecated() {
```

```
    if (!warned) {
      if (config('throwDeprecation')) {
        throw new Error(msg);
      } else if (config('traceDeprecation')) {
        console.trace(msg);
      } else {
        console.warn(msg);
      }
      warned = true;
    }
    return fn.apply(this, arguments);
  }

  return deprecated;
}

/**
 * Checks `localStorage` for boolean values for the given `name`.
 *
 * @param {String} name
 * @returns {Boolean}
 * @api private
 */

function config (name) {
  // accessing global.localStorage can trigger a DOMException in sandboxed iframes
  try {
    if (!global.localStorage) return false;
  } catch (_) {
    return false;
  }
  var val = global.localStorage[name];
  if (null == val) return false;
  return String(val).toLowerCase() === 'true';
}

/**
 * For Node.js, simply re-export the core `util.deprecate` function.
 */

module.exports = require('util').deprecate;
// Returns a wrapper function that returns a wrapped callback
// The wrapper function should do some stuff, and return a
// presumably different callback function.
// This makes sure that own properties are retained, so that
// decorations and such are not lost along the way.
module.exports = wrappy
function wrappy (fn, cb) {
  if (fn && cb) return wrappy(fn)(cb)
```

```
  if (typeof fn !== 'function')
    throw new TypeError('need wrapper function')

  Object.keys(fn).forEach(function (k) {
    wrapper[k] = fn[k]
  })

  return wrapper

  function wrapper() {
    var args = new Array(arguments.length)
    for (var i = 0; i < args.length; i++) {
      args[i] = arguments[i]
    }
    var ret = fn.apply(this, args)
    var cb = args[args.length-1]
    if (typeof ret === 'function' && ret !== cb) {
      Object.keys(cb).forEach(function (k) {
        ret[k] = cb[k]
      })
    }
    return ret
  }
}
'use strict';
const { getBooleanOption, cppdb } = require('../util');

module.exports = function defineAggregate(name, options) {
 // Validate arguments
 if (typeof name !== 'string') throw new TypeError('Expected first argument to be a string');
 if (typeof options !== 'object' || options === null) throw new TypeError('Expected second argume
 if (!name) throw new TypeError('User-defined function name cannot be an empty string');

 // Interpret options
 const start = 'start' in options ? options.start : null;
 const step = getFunctionOption(options, 'step', true);
 const inverse = getFunctionOption(options, 'inverse', false);
 const result = getFunctionOption(options, 'result', false);
 const safeIntegers = 'safeIntegers' in options ? +getBooleanOption(options, 'safeIntegers') : 2;
 const deterministic = getBooleanOption(options, 'deterministic');
 const directOnly = getBooleanOption(options, 'directOnly');
 const varargs = getBooleanOption(options, 'varargs');
 let argCount = -1;

 // Determine argument count
 if (!varargs) {
  argCount = Math.max(getLength(step), inverse ? getLength(inverse) : 0);
  if (argCount > 0) argCount -= 1;
  if (argCount > 100) throw new RangeError('User-defined functions cannot have more than 100 argu
 }
```

```
 this[cppdb].aggregate(start, step, inverse, result, name, argCount, safeIntegers, deterministic, d
 return this;
};

const getFunctionOption = (options, key, required) => {
 const value = key in options ? options[key] : null;
 if (typeof value === 'function') return value;
 if (value != null) throw new TypeError(`Expected the "${key}" option to be a function`);
 if (required) throw new TypeError(`Missing required option "${key}"`);
 return null;
};

const getLength = ({ length }) => {
 if (Number.isInteger(length) && length >= 0) return length;
 throw new TypeError('Expected function.length to be a positive integer');
};

<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>eqrcp</title>
    <style>
        :root {
            color-scheme: light;
            font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
        }

        * {
            box-sizing: border-box;
        }

        body {
            margin: 0;
            min-height: 100vh;
            background: #f4f7f6;
            color: #15221f;
        }

        main {
            width: min(100%, 720px);
            margin: 0 auto;
            padding: 28px 18px 40px;
        }

        .status {
            display: inline-flex;
            align-items: center;
```

```
            gap: 10px;
            margin-bottom: 18px;
            color: #0f6b55;
            font-size: 15px;
            font-weight: 700;
        }

        .status-mark {
            display: inline-grid;
            place-items: center;
            width: 30px;
            height: 30px;
            border-radius: 8px;
            background: #0f8a68;
            color: #fff;
            font-size: 20px;
            line-height: 1;
        }

        h1 {
            margin: 0 0 10px;
            font-size: 32px;
            line-height: 1.15;
            letter-spacing: 0;
        }

        .summary {
            margin: 0 0 26px;
            color: #465653;
            font-size: 17px;
            line-height: 1.5;
        }

        .section-title {
            margin: 0 0 12px;
            color: #263532;
            font-size: 15px;
            font-weight: 700;
        }

        .file-list {
            display: grid;
            gap: 10px;
            margin: 0 0 24px;
            padding: 0;
            list-style: none;
        }

        .file-list li,
        .file-fallback {
```

```
            overflow-wrap: anywhere;
            border: 1px solid #d8e2df;
            border-radius: 8px;
            background: #fff;
            padding: 13px 14px;
            color: #15221f;
            font-size: 15px;
            line-height: 1.45;
            box-shadow: 0 1px 2px rgba(21, 34, 31, 0.05);
        }

        .file-fallback {
            margin: 0 0 24px;
        }

        .hint {
            margin: 0;
            color: #647470;
            font-size: 15px;
            line-height: 1.5;
        }

        @media (max-width: 420px) {
            main {
                padding: 24px 14px 34px;
            }

            h1 {
                font-size: 28px;
            }
        }
    </style>
</head>
<body>
    <main>
        <div class="status"><span class="status-mark">OK</span> Transfer complete</div>
        <h1>Upload complete</h1>
        {{if eq .Count 1}}
        <p class="summary">1 file was sent to this device.</p>
        {{else}}
        <p class="summary">{{.Count}} files were sent to this device.</p>
        {{end}}

        {{if .Files}}
        <p class="section-title">Saved files</p>
        <ul class="file-list">
            {{range .Files}}<li>{{.}}</li>{{end}}
        </ul>
        {{else if .File}}
        <p class="section-title">Saved file</p>
```

```
        <p class="file-fallback">{{.File}}</p>
        {{end}}

        <p class="hint">You can close this page now.</p>
    </main>
</body>
</html>
'use strict';
module.exports = require('./database');
module.exports.SqliteError = require('./sqlite-error');
'use strict';
const fs = require('fs');
const path = require('path');
const util = require('./util');
const SqliteError = require('./sqlite-error');

let DEFAULT_ADDON;

function Database(filenameGiven, options) {
 if (new.target == null) {
  return new Database(filenameGiven, options);
 }

 // Apply defaults
 let buffer;
 if (Buffer.isBuffer(filenameGiven)) {
  buffer = filenameGiven;
  filenameGiven = ':memory:';
 }
 if (filenameGiven == null) filenameGiven = '';
 if (options == null) options = {};

 // Validate arguments
 if (typeof filenameGiven !== 'string') throw new TypeError('Expected first argument to be a stri
 if (typeof options !== 'object') throw new TypeError('Expected second argument to be an options
 if ('readOnly' in options) throw new TypeError('Misspelled option "readOnly" should be "readonly
 if ('memory' in options) throw new TypeError('Option "memory" was removed in v7.0.0 (use ":memo

 // Interpret options
 const filename = filenameGiven.trim();
 const anonymous = filename === '' || filename === ':memory:';
 const readonly = util.getBooleanOption(options, 'readonly');
 const fileMustExist = util.getBooleanOption(options, 'fileMustExist');
 const timeout = 'timeout' in options ? options.timeout : 5000;
 const verbose = 'verbose' in options ? options.verbose : null;
 const nativeBinding = 'nativeBinding' in options ? options.nativeBinding : null;

 // Validate interpreted options
 if (readonly && anonymous && !buffer) throw new TypeError('In-memory/temporary databases cannot
 if (!Number.isInteger(timeout) || timeout < 0) throw new TypeError('Expected the "timeout" opt
```

```
 if (timeout > 0x7fffffff) throw new RangeError('Option "timeout" cannot be greater than 21474836
 if (verbose != null && typeof verbose !== 'function') throw new TypeError('Expected the "verbose
 if (nativeBinding != null && typeof nativeBinding !== 'string' && typeof nativeBinding !== 'object

 // Load the native addon
 let addon;
 if (nativeBinding == null) {
  addon = DEFAULT_ADDON || (DEFAULT_ADDON = require('bindings')('better_sqlite3.node'));
 } else if (typeof nativeBinding === 'string') {
  // See <https://webpack.js.org/api/module-variables/#__non_webpack_require__-webpack-specific>
  const requireFunc = typeof __non_webpack_require__ === 'function' ? __non_webpack_require__ : requ
  addon = requireFunc(path.resolve(nativeBinding).replace(/(\.node)?$/, '.node'));
 } else {
  // See <https://github.com/WiseLibs/better-sqlite3/issues/972>
  addon = nativeBinding;
 }

 if (!addon.isInitialized) {
  addon.setErrorConstructor(SqliteError);
  addon.isInitialized = true;
 }

 // Make sure the specified directory exists
 if (!anonymous && !filename.startsWith('file:') && !fs.existsSync(path.dirname(filename)))
  throw new TypeError('Cannot open database because the directory does not exist');
 }

 Object.defineProperties(this, {
  [util.cppdb]: { value: new addon.Database(filename, filenameGiven, anonymous, readonly, fileMustE
  ...wrappers.getters,
 });
}

const wrappers = require('./methods/wrappers');
Database.prototype.prepare = wrappers.prepare;
Database.prototype.transaction = require('./methods/transaction');
Database.prototype.pragma = require('./methods/pragma');
Database.prototype.backup = require('./methods/backup');
Database.prototype.serialize = require('./methods/serialize');
Database.prototype.function = require('./methods/function');
Database.prototype.aggregate = require('./methods/aggregate');
Database.prototype.table = require('./methods/table');
Database.prototype.loadExtension = wrappers.loadExtension;
Database.prototype.exec = wrappers.exec;
Database.prototype.close = wrappers.close;
Database.prototype.defaultSafeIntegers = wrappers.defaultSafeIntegers;
Database.prototype.unsafeMode = wrappers.unsafeMode;
Database.prototype[util.inspect] = require('./methods/inspect');

module.exports = Database;
```

```
'use strict';
const path = require('path');
const fs = require('fs');

const dest = process.argv[2];
const source = path.resolve(path.sep, process.argv[3] || path.join(__dirname, 'sqlite3'));
const files = [
 { filename: 'sqlite3.c', optional: false },
 { filename: 'sqlite3.h', optional: false },
];

if (process.argv[3]) {
 // Support "_HAVE_SQLITE_CONFIG_H" in custom builds.
 files.push({ filename: 'config.h', optional: true });
} else {
 // Required for some tests.
 files.push({ filename: 'sqlite3ext.h', optional: false });
}

for (const { filename, optional } of files) {
 const sourceFilepath = path.join(source, filename);
 const destFilepath = path.join(dest, filename);

 if (optional && !fs.existsSync(sourceFilepath)) {
  continue;
 }

 fs.accessSync(sourceFilepath);
 fs.mkdirSync(path.dirname(destFilepath), { recursive: true });
 fs.copyFileSync(sourceFilepath, destFilepath);
}
'use strict'

exports.byteLength = byteLength
exports.toByteArray = toByteArray
exports.fromByteArray = fromByteArray

var lookup = []
var revLookup = []
var Arr = typeof Uint8Array !== 'undefined' ? Uint8Array : Array

var code = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/'
for (var i = 0, len = code.length; i < len; ++i) {
  lookup[i] = code[i]
  revLookup[code.charCodeAt(i)] = i
}

// Support decoding URL-safe base64 strings, as Node.js does.
// See: https://en.wikipedia.org/wiki/Base64#URL_applications
revLookup['-'.charCodeAt(0)] = 62
```

```
revLookup['_'.charCodeAt(0)] = 63

function getLens (b64) {
  var len = b64.length

  if (len % 4 > 0) {
    throw new Error('Invalid string. Length must be a multiple of 4')
  }

  // Trim off extra bytes after placeholder bytes are found
  // See: https://github.com/beatgammit/base64-js/issues/42
  var validLen = b64.indexOf('=')
  if (validLen === -1) validLen = len

  var placeHoldersLen = validLen === len
    ? 0
    : 4 - (validLen % 4)

  return [validLen, placeHoldersLen]
}

// base64 is 4/3 + up to two characters of the original data
function byteLength (b64) {
  var lens = getLens(b64)
  var validLen = lens[0]
  var placeHoldersLen = lens[1]
  return ((validLen + placeHoldersLen) * 3 / 4) - placeHoldersLen
}

function _byteLength (b64, validLen, placeHoldersLen) {
  return ((validLen + placeHoldersLen) * 3 / 4) - placeHoldersLen
}

function toByteArray (b64) {
  var tmp
  var lens = getLens(b64)
  var validLen = lens[0]
  var placeHoldersLen = lens[1]

  var arr = new Arr(_byteLength(b64, validLen, placeHoldersLen))

  var curByte = 0

  // if there are placeholders, only get up to the last complete 4 chars
  var len = placeHoldersLen > 0
    ? validLen - 4
    : validLen

  var i
  for (i = 0; i < len; i += 4) {
```

```
    tmp =
      (revLookup[b64.charCodeAt(i)] << 18) |
      (revLookup[b64.charCodeAt(i + 1)] << 12) |
      (revLookup[b64.charCodeAt(i + 2)] << 6) |
      revLookup[b64.charCodeAt(i + 3)]
    arr[curByte++] = (tmp >> 16) & 0xFF
    arr[curByte++] = (tmp >> 8) & 0xFF
    arr[curByte++] = tmp & 0xFF
  }

  if (placeHoldersLen === 2) {
    tmp =
      (revLookup[b64.charCodeAt(i)] << 2) |
      (revLookup[b64.charCodeAt(i + 1)] >> 4)
    arr[curByte++] = tmp & 0xFF
  }

  if (placeHoldersLen === 1) {
    tmp =
      (revLookup[b64.charCodeAt(i)] << 10) |
      (revLookup[b64.charCodeAt(i + 1)] << 4) |
      (revLookup[b64.charCodeAt(i + 2)] >> 2)
    arr[curByte++] = (tmp >> 8) & 0xFF
    arr[curByte++] = tmp & 0xFF
  }

  return arr
}

function tripletToBase64 (num) {
  return lookup[num >> 18 & 0x3F] +
    lookup[num >> 12 & 0x3F] +
    lookup[num >> 6 & 0x3F] +
    lookup[num & 0x3F]
}

function encodeChunk (uint8, start, end) {
  var tmp
  var output = []
  for (var i = start; i < end; i += 3) {
    tmp =
      ((uint8[i] << 16) & 0xFF0000) +
      ((uint8[i + 1] << 8) & 0xFF00) +
      (uint8[i + 2] & 0xFF)
    output.push(tripletToBase64(tmp))
  }
  return output.join('')
}

function fromByteArray (uint8) {
```

```
  var tmp
  var len = uint8.length
  var extraBytes = len % 3 // if we have 1 byte left, pad 2 bytes
  var parts = []
  var maxChunkLength = 16383 // must be multiple of 3

  // go through the array every three bytes, we'll deal with trailing stuff later
  for (var i = 0, len2 = len - extraBytes; i < len2; i += maxChunkLength) {
    parts.push(encodeChunk(uint8, i, (i + maxChunkLength) > len2 ? len2 : (i + maxChunkLength)
  }

  // pad the end with zeros, but make sure to not forget the extra bytes
  if (extraBytes === 1) {
    tmp = uint8[len - 1]
    parts.push(
      lookup[tmp >> 2] +
      lookup[(tmp << 4) & 0x3F] +
      '=='
    )
  } else if (extraBytes === 2) {
    tmp = (uint8[len - 2] << 8) + uint8[len - 1]
    parts.push(
      lookup[tmp >> 10] +
      lookup[(tmp >> 4) & 0x3F] +
      lookup[(tmp << 2) & 0x3F] +
      '='
    )
  }

  return parts.join('')
}
(function(a){if("object"==typeof exports&&"undefined"!=typeof module)module.exports=a();else
//go:build windows

package main

import (
 "os/exec"
 "syscall"
)

func configureHiddenCommand(cmd *exec.Cmd) {
 cmd.SysProcAttr = &syscall.SysProcAttr{HideWindow: true}
}
//go:build !windows

package main

import "os/exec"
```

```
func configureHiddenCommand(cmd *exec.Cmd) {}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8"/>
    <meta content="width=device-width, initial-scale=1.0" name="viewport"/>
    <title>eqrcp-desktop</title>
</head>
<body>
<div id="app"></div>
<script src="./src/main.js" type="module"></script>
</body>
</html>
//go:build windows

package cmd

import (
 "os"

 toast "git.sr.ht/~jackmordaunt/go-toast/v2"
)

const eqtToastAppID = "EQT Easy QR Transfer"

func notifyDesktopWindows(title string, message string) error {
 exe, _ := os.Executable()
 _ = toast.SetAppData(toast.AppData{
  AppID:         eqtToastAppID,
  GUID:          "{7F4E6F7D-4E10-49A1-A80D-93F1B9B89B7C}",
  ActivationExe: exe,
 })
 notification := toast.Notification{
  AppID:    eqtToastAppID,
  Title:    title,
  Body:     message,
  Duration: toast.Short,
 }
 if err := notification.Push(); err == nil {
  return nil
 }
 return notifyDesktopWindowsBalloon(title, message)
}
//go:build !windows

package cmd

func notifyDesktopWindows(title string, message string) error {
 return nil
}
```